

SQL One to One One to Many Many to Many

8 Temmuz 2025 Salı 10:02

☑️ 1. One-to-One (Bire Bir) İlişki

Senaryo: Her kullanıcının bir kimliği vardır (User ↔ IdentityCard)

```
public class User
{
    public int UserId { get; set; }
    public string Name { get; set; }

    public IdentityCard IdentityCard { get; set; } // One-to-One
}

public class IdentityCard
{
    public int IdentityCardId { get; set; }
    public string CardNumber { get; set; }

    public int UserId { get; set; }
    public User User { get; set; }
}
```

❑ Fluent API (Opsiyonel ama önerilir)

```
modelBuilder.Entity<User>()
    .HasOne(u => u.IdentityCard)
    .WithOne(i => i.User)
    .HasForeignKey<IdentityCard>(i => i.UserId);
```

☑️ 2. One-to-Many (Bire Çok) İlişki

Senaryo: Bir yazarın birçok kitabı olabilir (Author → Books)

```
public class Author
{
    public int AuthorId { get; set; }
    public string Name { get; set; }
    public List<Book> Books { get; set; } // One-to-Many
}

public class Book
{
    public int BookId { get; set; }
    public string Title { get; set; }
    public int AuthorId { get; set; }
    public Author Author { get; set; }
}
```

❑ Fluent API (Gerekmez ama özelleştirilebilir)

```
modelBuilder.Entity<Book>()
    .HasOne(b => b.Author)
    .WithMany(a => a.Books)
    .HasForeignKey(b => b.AuthorId);
```

☑️ 3. Many-to-Many (Çoktan Çoğa) İlişki

Senaryo: Öğrenciler birçok kurs alabilir, kurslar birçok öğrenci içerebilir.

```
public class Student
{
    public int StudentId { get; set; }
    public string FullName { get; set; }

    public List<Course> Courses { get; set; }
}

public class Course
{
    public int CourseId { get; set; }
    public string Title { get; set; }

    public List<Student> Students { get; set; }
}
```

❑ EF Core 5+ Fluent API (otomatik Many-to-Many destekli)

```
modelBuilder.Entity<Student>()
    .HasMany(s => s.Courses)
    .WithMany(c => c.Students);
```

```
}
```

Ek Notlar

.OnDelete(DeleteBehavior.Cascade);

Ana kayıt silinince alt kayıtlar da silinsin Cascade

Ana kayıt silinmeden alt kayıtlar silinmeli Restrict

Ana kayıt silinince FK null olsun SetNull

EF tarafı silsin ama DB tarafı sessiz kalsın ClientSetNull

□ 1. One-to-One (Bire Bir)

```
modelBuilder.Entity<User>()
```

```
.HasOne(u => u.IdentityCard) // User bir IdentityCard'a sahiptir
```

```
.WithOne(i => i.User) // IdentityCard da bir User'a sahiptir
```

```
.HasForeignKey<IdentityCard>(i => i.UserId);
```

□ 2. One-to-Many (Bire Çok)

```
modelBuilder.Entity<Author>()
```

```
.HasMany(a => a.Books) // Author'ın birçok kitabı vardır
```

```
.WithOne(b => b.Author) // Her kitap bir yazara aittir
```

```
.HasForeignKey(b => b.AuthorId);
```

□ 3. Many-to-Many (Çoktan Çoğa)

```
modelBuilder.Entity<Student>()
```

```
.HasMany(s => s.Courses)
```

```
.WithMany(c => c.Students);
```